Academic Year: 2026-2027
Semester: III
Class / Branch: SE IT
Subject:ADSAL lab
Name of Instructor:prof.Sneha Dalvi

Name of Student:Nishant Suryarao
Student ID:26204001
Date of Performance:08/10/2026
Date of Submission:08/10/2026

Experiment No.10 Aim:

Implement Travelling salesman problem.

Code:

```
#include <stdio.h> #define INF
9999 int n; // number of cities int
cost[20][20]; // adjacency matrix
int visited[20]; // visited array

int tsp(int city, int count, int dist) { if
(count == n && cost[city][0]) {
return dist + cost[city][0]; // return to start
}

int ans = INF; for (int i = 0; i < n;
i++) { if (!visited[i] &&
cost[city][i]) { visited[i] = 1;
int newAns = tsp(i, count + 1, dist + cost[city][i]);
if (newAns < ans) ans = newAns;
visited[i] = 0; // backtrack
}}
return ans;
}
int main() {
printf("Enter number of cities: "); scanf("%d",
&n); printf("Enter cost matrix (use 0 if no
```

```
path):\n"); for (int i = 0; i < n; i++) { for (int j =
0; j < n; j++) {
scanf("%d", &cost[i][j]);
}
}

for (int i = 0; i < n; i++) visited[i] = 0; visited[0] =
1; // start from city 0

int ans = tsp(0, 1, 0); printf("Minimum cost of travelling
salesman tour = %d\n", ans);

return 0;
}
```

Output Screen Shots:

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ gcc -o Exp10 Exp10.c
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ ./Exp10
Enter number of cities: 2
Enter cost matrix (use 0 if no path):
2 5
6 5
Minimum cost of travelling salesman tour = 11
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$
```

Conclusion :

The Travelling Salesman Problem was successfully implemented using backtracking. The algorithm explored possible routes and determined the minimum-cost tour that visits every city exactly once and returns to the starting city.

Experiment 10: Travelling Salesman Problem (TSP)

Definition: The Travelling Salesman Problem finds the minimum-cost tour in which a salesman visits every city exactly once and returns to the starting city.

Algorithm:

1. Start.
2. Read the number of cities and the cost matrix.
3. Start from city 0 and mark it as visited.

4. Consider each unvisited city connected to the current city.
5. Mark the selected city as visited and recursively explore the next city.
6. When all cities are visited, add the cost of returning to the starting city.
7. Compare the costs of all possible valid tours.
8. Display the minimum tour cost.
9. Stop.

Conclusion: Thus, the Travelling Salesman Problem was successfully implemented using backtracking to determine the minimum-cost tour.
Complexity: Worst-case time \(O(n!)\).